

Fundamental of Computer Science

Linux and GCC Study Notes

These notes combine the five supplied lab manuals into one study sequence. The two Practical 1 files repeat the Linux installation and GCC material, so that content appears once. The later manuals add file and directory commands, command help, user identity, privilege, and system status. Examples are normalized for a Linux shell; check your distribution's manual pages when options vary.

1. Linux and a Virtual Machine

Linux is an open-source, Unix-like operating system kernel. A Linux distribution combines the kernel with system tools, libraries, package management, and applications. Distributions such as Ubuntu, Fedora, and Debian package these components differently while sharing many command-line conventions.

VirtualBox lets a Linux guest run inside a virtual machine (VM) on a host computer. The guest has virtual hardware and its own virtual disk. The basic lab workflow is:

- Install Oracle VirtualBox on the host and download a Linux ISO from the distribution's official site.
- Create a VM, select Linux and the matching distribution version, and allocate RAM, CPU cores, and a virtual disk while leaving enough resources for the host.
- Attach the ISO to the VM's virtual optical drive, start the VM, and follow the installer for language, keyboard, time zone, disk, and initial account settings.
- Eject the ISO after installation, restart the VM, verify that Linux starts, then install VirtualBox Guest Additions if clipboard integration and dynamic display sizing are needed.

Treat the VM disk as the guest's storage. Choose disk and memory sizes appropriate to the host; the source manuals do not specify fixed values.

2. GCC and Building C Programs

GCC (GNU Compiler Collection) supports C, C++, and other languages. For C, `gcc` drives several stages: preprocessing expands directives and headers; compilation translates C into assembly; the assembler creates object files; and the linker combines object files and libraries into an executable. `g++` is the usual driver for C++ programs.

Check which tools are installed:

```
gcc --version
g++ --version
make --version
```

On Debian or Ubuntu, the common package bundle is:

```
sudo apt update
sudo apt install build-essential
```

To compile and run a small C program, save it as `hello.c`:

```
#include <stdio.h>

int main(void) {
    puts("Hello");
    return 0;
}
```

Then build and run it:

```
gcc -Wall -o hello hello.c
./hello
echo $?
```

`-Wall` enables a useful set of warnings, `-o hello` names the executable, `./hello` runs a file in the current directory, and `$?` displays the last command's exit status (zero normally means success).

GCC can also stop after intermediate stages. These are useful for learning or debugging:

```
gcc -E hello.c -o hello.i    # preprocessed source
gcc -S hello.c -o hello.s    # assembly source
gcc -c hello.c -o hello.o    # object file; no link step
gcc -Wall -o hello hello.c   # compile and link to an executable
```

A small multi-file project

The Practical 1 example divides arithmetic functions across source files. A header declares the functions; each `.c` file defines one part; the linker combines the resulting object files.

`calc.h`:

```
int add(int a, int b);
int sub(int a, int b);
int mul(int a, int b);
```

`main.c`:

```
#include <stdio.h>
#include "calc.h"

int main(void) {
    printf("sum is %d\n", add(3, 4));
    printf("sub is %d\n", sub(3, 2));
    printf("mul is %d\n", mul(3, 2));
    return 0;
}
```

Each function definition belongs in its own source file (`add.c`, `sub.c`, and `mul.c`), including `calc.h` and returning the appropriate arithmetic result. Compile and link the project directly with:

```
gcc -Wall -o calculator main.c add.c sub.c mul.c
./calculator
```

A Makefile records the dependencies so `make` can rebuild what changed. Recipe lines must begin with a tab:

```
calculator: main.o add.o sub.o mul.o
\tgcc -o calculator main.o add.o sub.o mul.o

main.o: main.c calc.h
\tgcc -Wall -c main.c

add.o: add.c calc.h
\tgcc -Wall -c add.c

sub.o: sub.c calc.h
\tgcc -Wall -c sub.c

mul.o: mul.c calc.h
\tgcc -Wall -c mul.c

.PHONY: clean
clean:
\t rm -f *.o calculator
```

Run **make** to build the default target, then **./calculator**. Run **make clean** to remove generated object files and the executable. **rm -f** is limited here by the explicit filename pattern; avoid broad deletion patterns outside a disposable project directory.

3. Paths, Directories, and Files

The shell starts in a current working directory. An absolute path begins at **/**; a relative path is interpreted from the current directory. **~** expands to the current user's home directory. Use **pwd** to see where you are and **ls** to inspect a directory.

```
pwd
ls -lah
cd ~/linux_lab
cd ..
cd -
```

cd is a shell built-in. **cd** or **cd ~** goes home, **cd ..** moves to the parent, **cd -** returns to the previous directory, and **cd /** changes to the filesystem root. **pwd -L** displays the logical path (which may contain symbolic links); **pwd -P** resolves it to the physical path.

Create a nested practice workspace and files:

```
mkdir -p ~/linux_lab/docs
cd ~/linux_lab
touch notes.txt
echo "Linux command line practice" > notes.txt
echo "A second line" >> notes.txt
```

mkdir -p creates missing parent directories. A single **>** replaces a file's contents; **>>** appends. **touch** creates an empty file if it does not exist and updates its timestamp if it does.

Copy, move, and rename files with **cp** and **mv**:

```
cp -v notes.txt docs/notes-copy.txt
mv -v notes.txt linux-notes.txt
```

Use **cp -r** to copy a directory tree. **-i** asks before overwriting; **-n** prevents overwriting on GNU systems; **-v** reports each operation. **mv** moves a path or renames it when the destination is in the same directory.

Remove empty directories with **rmdir**. To remove a non-empty practice tree, **rm -r** is recursive and permanent:

```
rmdir empty-folder
rm -ri ~/linux_lab
```

The `-i` flag prompts while removing items. `rm -rf` suppresses prompts and forces recursive removal; a wrong path can destroy valuable data. Check the path with `pwd` and `ls` before removal. Never use an uncertain variable or wildcard with recursive deletion.

Read text files

`cat` prints a whole file, so it is best for short files. `less` opens a pager for longer files; press `q` to exit, `/word` to search forward, `n` for the next match, and `N` for the previous match.

```
cat -n notes.txt
less -N /etc/passwd
```

`cat -n` numbers all lines; `cat -b` numbers non-empty lines; `cat -E` marks line endings; `cat -s` squeezes repeated blank lines.

4. Finding Commands and Help

Use `whereis` to locate a command's binary, source, and manual page; use `whatis` for a one-line manual summary:

```
whereis ls
whereis -b gcc
whatis cp mv rm
```

If `whatis` has no indexed result on a Debian/Ubuntu system, update the manual-page database with `sudo mandb` (where available). `man` command opens the full manual; `man -k keyword` searches manual summaries. `help` describes Bash built-ins such as `cd`, and `history` displays recent commands.

5. Users, Privileges, and System Status

These commands describe the current account and active sessions:

```
whoami
id
who
tty
```

`whoami` prints the effective username. `id` shows user and group IDs; `id -un` prints the effective username. `who` lists logged-in sessions and `who -b` reports the last boot. `tty` prints the terminal device connected to the current shell.

`su` starts a shell as another user; `su - username` also loads that user's login environment. `sudo` command runs one command with privileges allowed by system policy, usually asking for the current user's password. Prefer a single `sudo` command when needed; elevated commands can alter the whole system. `sudo -l` lists allowed commands, `sudo -i` starts an interactive root login shell, and `exit` or `Ctrl+D` ends the elevated shell. Do not create users or change passwords except on a system where you have permission.

Inspect the system and resources:

```
uname -a
uptime
free -h
hostname
date
cal
```

`uname -a` summarizes kernel and machine information. `uptime` reports running time, logged-in users, and load averages. `free -h` presents memory and swap totals in readable units. `hostname -I` lists configured IP addresses on many Linux systems. `date -u` shows UTC time; formatting can be requested with `date +%Y-%m-%d %H:%M:%S`. `cal -y` displays the current year's calendar.

`clear` clears the terminal display. `reboot` restarts the system and generally requires administrator privileges. Use it only when appropriate and after saving work. `history -c` clears the current shell's history list; it does not erase shell history from other sessions or logs.

6. A Short Practice Sequence

Try the following in a temporary folder in your home directory. Inspect each result before moving to the next step:

```
mkdir -p ~/linux_lab/docs
cd ~/linux_lab
printf "Line 1\nLine 2\n" > notes.txt
cp -i notes.txt docs/notes-copy.txt
mv -i notes.txt linux-notes.txt
cat -n linux-notes.txt
less docs/notes-copy.txt
cd ~
```

When finished, clean up only the practice folder after confirming its path:

```
pwd
ls -la ~/linux_lab
rm -ri ~/linux_lab
```

Source Coverage

Prepared from [Lab manual of Fundamental of Computer Science](#), [Lab manual 1 of FCS](#), and [Lab Manuals 2–4](#). The two Practical 1 documents repeat most of the same setup and GCC material. The notes retain their shared concepts, add the later Linux-command labs, and standardize malformed command examples for use in a Linux shell.